

Network File Sharing Notes

1. File Sharing Overview

Network file sharing allows data transfer between machines on the same network without physical media like USB drives. This is essential in commercial environments where multiple computers need to share data efficiently.

Key Methods Covered:

- Simple file copies between hosts
 - Mounting entire directories as separate drives
 - Various protocols and tools for different use cases
-

2. SCP (Secure Copy)

Description: A secure file transfer tool that works like the `cp` command but operates over SSH for network transfers.

Key Features:

- Uses SSH authentication and security
- Simple syntax similar to local copy commands
- Supports both file and directory transfers

Examples:

```
# Copy file from local to remote host
$ scp myfile.txt username@remotehost.com:/remote/directory

# Copy file from remote to local host
$ scp username@remotehost.com:/remote/directory/myfile.txt /local/directory

# Copy directory recursively from local to remote host
$ scp -r mydir username@remotehost.com:/remote/directory
```

3. Rsync (Remote Synchronization)

Description: Advanced copying tool that only transfers differences between files, making it more efficient than scp for large or repeated transfers.

Key Features:

- Special algorithm checks for existing data
- Only copies differences (delta sync)
- Verifies file integrity with checksums
- Ideal for backups, large transfers, and directory synchronization
- Can resume interrupted transfers

Common Options:

- `v` - verbose output
- `r` - recursive into directories
- `h` - human readable output
- `z` - compressed transfer (good for slow connections)

Examples:

```
# Copy/sync files on same host
$ rsync -zvr /my/local/directory/one /my/local/directory/two

# Copy/sync from remote to local
$ rsync /local/directory username@remotehost.com:/remote/directory

# Copy/sync from local to remote
$ rsync username@remotehost.com:/remote/directory /local/directory
```

4. Simple HTTP Server

Description: Python's built-in HTTP server for quick file sharing over the network.

Use Case: Creating temporary network shares accessible via web browser

Example:

```
# Start HTTP server in current directory
$ python -m SimpleHTTPServer
```

```
# Access from other machines
http://IP_ADDRESS:8000

# Access locally
http://localhost:8000
```

Note: Python 3 and Node.js have different syntax for similar functionality.

5. NFS (Network File System)

Description: Standard Linux network file sharing protocol allowing servers to share directories with multiple clients.

Key Features:

- Standard for Linux environments
- Allows mounting remote directories as local filesystems
- Supports automounting for on-demand connections

Client Setup Examples:

```
# Start NFS client service
$ sudo service nfsclient start

# Mount NFS share
$ sudo mount server:/directory /mount_directory
```

Automounting:

- Uses `automount` tool (or `amd` in recent Linux versions)
 - Automatically mounts when files are accessed
 - Prevents boot issues when server is unavailable
 - Better than permanent `/etc/fstab` entries for unreliable connections
-

6. Samba (SMB/CIFS)

Description: Linux implementation of Windows file sharing protocols (SMB/CIFS) for cross-platform compatibility.

Key Features:

- Enables file sharing between Windows and Linux
- Also supports printer sharing
- Uses SMB (Server Message Block) and CIFS (Common Internet File System) protocols

Setting up Samba Server

```
# Install Samba
$ sudo apt update
$ sudo apt install samba

# Edit configuration file
$ sudo vi /etc/samba/smb.conf

# Set up Samba password
$ sudo smbpasswd -a [username]

# Create shared directory
$ mkdir /my/directory/to/share

# Restart Samba service
$ sudo service smbd restart
```

Accessing Samba Shares

From Windows:

```
\\HOST\sharename
```

(Enter in Windows Run prompt)

From Linux:

```
# Access via smbclient
$ smbclient //HOST/directory -U user

# Mount Samba share
$ sudo mount -t cifs servername:directory mountpoint -o
user=username,pass=password
```

Summary

Network file sharing provides multiple solutions for different scenarios:

- **SCP**: Simple, secure transfers using SSH
- **Rsync**: Efficient synchronization with delta transfers
- **HTTP Server**: Quick temporary sharing via web browser
- **NFS**: Standard Linux network filesystem
- **Samba**: Cross-platform Windows/Linux compatibility

Choose the appropriate method based on your security requirements, transfer frequency, and client operating systems.